Faculty of Engineering and Technology
B.Tech (CSE) Programme
Semester V
Subject: Artificial Intelligence [2601504]
MODULE 1

Heuristic Search Techniques: Generate-and-Test, Hill Climbing, Best-first Search, Problem Reduction, Constraint Satisfaction, Means-ends

Uninformed / Informed Search

Feature	Uninformed Search (Blind Search)	Informed Search (Heuristic Search)
Definition	Searches without extra information.	Searches using heuristic (extra information).
Knowledge Used	Uses only the initial state, goal state, and rules.	Uses heuristic function **h(n)** to estimate the best path.
How It Works	Explores nodes in a fixed order.	Explores the most promising node first (algorithm selects the node that is most likely to reach the goal quickly instead of checking every possible node.).
Decision Making	Does not know which path is better.	Chooses the path that seems closest to the goal.
Heuristic Used	❌ No	✅ Yes
Speed	Slower	Faster
Nodes Explored	More nodes	Fewer nodes
Efficiency	Less efficient	More efficient
Advantage	Simple and easy to implement.	Faster and reaches the goal quickly.
Disadvantage	May waste time exploring unnecessary nodes.	Depends on the quality of the heuristic.
Real-Life Example	Searching every classroom to find one room.	Using a college map to reach the classroom directly.

Examples	BFS, DFS, Uniform Cost Search	Best-First Search, Greedy Search, A* Search, Hill Climbing

Uninformed Search	Informed Search
No clue → Search blindly	**Has a clue → Search intelligently**
More exploration	Less exploration
Slower	Faster
No heuristic	Uses heuristic

Heuristic

Heuristic = Intelligent Guess

A **heuristic** is an **intelligent guess or clue** that helps us choose the best path to reach the goal faster.

Example

Suppose you want to reach your classroom.

Your friend says:

- Left path → Looks shorter ✅
- Right path → Looks longer

You choose the **left path**.

Your friend's suggestion is called a **heuristic** because it helps you make a quick decision.

Heuristic Function

Heuristic Function (h(n)):
A function that gives an estimated value (distance or cost) from the current node to the goal.

Scenario

Suppose you want to reach your **college** from your home.

There are **three roads** available.

Road	Estimated Distance to College
Road A	8 km
Road B	3 km
Road C	5 km

Step 1: Heuristic

The **estimated distance** helps you decide which road is better.

Here,

- Road A → 8 km
- Road B → 3 km ✅
- Road C → 5 km

Since **Road B has the smallest estimated distance (3 km)**, you will choose **Road B**.

This estimated distance (3 km) is called the Heuristic Value, h(n).

Step 2: Heuristic Function

The **Heuristic Function** is the method that provides these estimated distances.

It gives:

- **h(A) = 8**
- **h(B) = 3**
- **h(C) = 5**

The search algorithm compares these values and selects the **smallest one**.

Generate-and-Test

Generate-and-Test is a heuristic search technique in which the algorithm:

1. Generates a possible solution.
2. Tests whether it is the correct solution. (Match with the goal state, accept, or else Reject)
3. If it is not correct, it generates another solution and repeats the process.

Formula:
Generate → Test → If correct, Stop; Otherwise, Repeat

Example

Suppose the password is 4567.

The computer tries:

- 1234 ❌
- 3456 ❌
- 4567 ✅

When it finds the correct password, it stops.

Example

Finding the correct key for a lock.

You try one key after another until the lock opens.

Applications

- Password guessing
- Puzzle solving
- Cryptography

Properties of a Good Generator

Property	Meaning	Example
Complete	The generator should be able to **generate all possible solutions** so that the correct solution is not missed.	If there are 5 keys, it should eventually try all 5 keys.

Non-Redundant	The generator should **not generate the same solution again and again.** This saves time and effort.	If Key 2 has already been tested, it should not test Key 2 again.
Informed	The generator should **try the best or most likely solution first** to find the answer faster.	If one road is estimated to take 5 minutes and another 20 minutes, it should try the 5-minute road first.

Hill Climbing

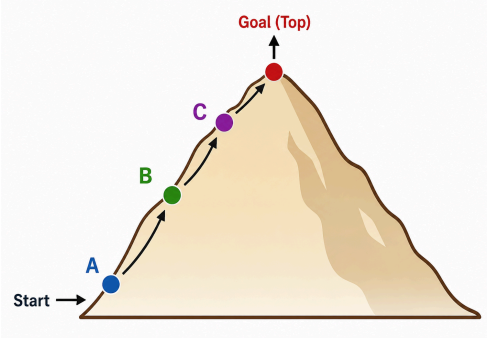
Hill Climbing is a **heuristic search technique** that **always moves to the better neighboring state** until it reaches the goal.

It does not check all possible paths. It only looks at the nearby options and chooses the best one.

Example

Think of it like climbing a hill in thick fog — you can't see the top, but you can feel which direction goes up. So at every step, you just move toward the direction that goes **higher**, and you keep doing that until you can't go any higher.

That's it. No looking ahead, no memory of past steps — just: check neighbors, pick the better one, repeat.



- You start at **A**.
- You look around and move to a **higher position (B)**.
- From **B**, you move to an even **higher position (C)**.
- Finally, you reach the **top of the hill (Goal)**.
  PATH: A → B → C → Goal

Step-by-Step Working

1. Start from any position.
2. Check the neighboring states.
3. Move to the neighbor with the highest value.
4. Repeat Steps 2 and 3.
5. Stop when no neighboring state has a better value.
6. The final state may be:
    - ✅ Global Maximum (Goal), or
    - ❌ Local Maximum (not the best solution).

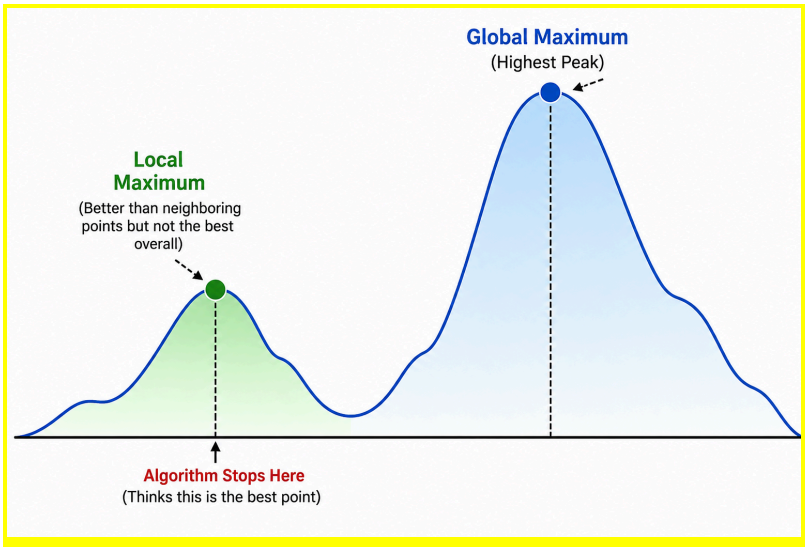
Problems with Hill Climbing

1. **Local Maximum (Stops at the wrong hill.)**

Hill Climbing keeps moving to a better neighboring state. It stops when there is no better neighbor. If the current state is the highest point, it reaches the goal. Otherwise, it may get stuck at a local maximum.

(Local Maximum: A solution that is better than all its neighboring solutions, but it is not the best solution overall. Hill Climbing may stop at a local maximum because it only checks nearby states and cannot see better solutions farther away.
Local Maximum = Best nearby solution 🏞️
Global Maximum = Best overall solution 🏔️)

A local maximum is a point that is higher than all its nearby points, but it is not the highest point overall.

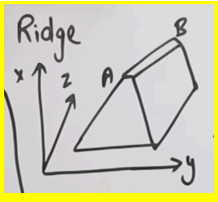


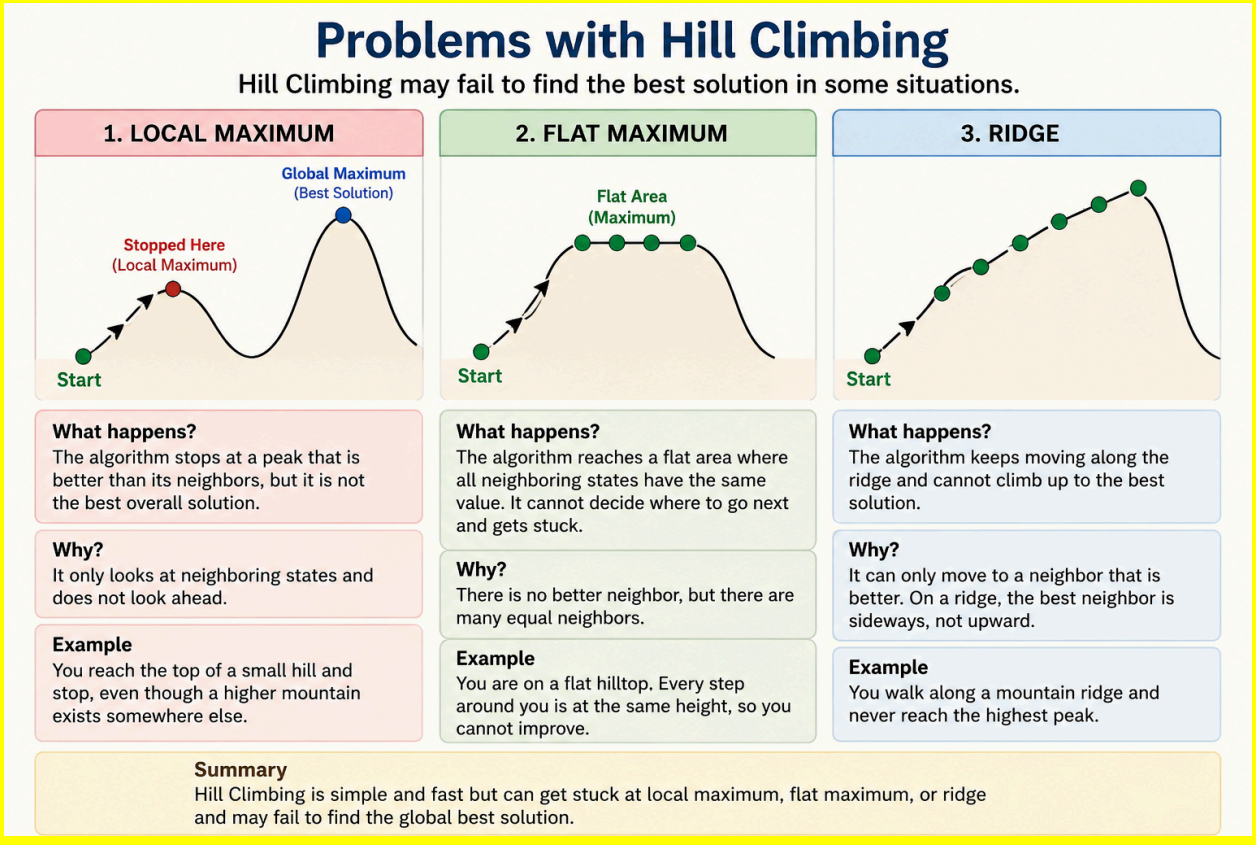
2. Flat Maximum (Gets confused on a flat area.)

   The algorithm reaches a **flat area** where all neighboring points have the **same value**. It cannot decide which direction to move, so it stops or moves randomly.

3. Ridge : Cannot move in the correct direction.

   The algorithm **cannot reach the best solution** because the correct path requires a direction it cannot easily follow.





Best-First Search (Greedy Best-First Search)

Best-First Search is an informed search algorithm that expands the node with the best heuristic value, aiming to reach the goal as quickly as possible.

- Best-First Search is an informed search algorithm that selects the most promising node to explore first.
- It uses a heuristic function (h(n)) to estimate how close a node is to the goal.
- The algorithm always expands the node with the lowest heuristic value, hoping to reach the goal quickly.
- Best-First Search is generally faster than uninformed search methods, but it does not always guarantee the shortest or optimal path.

Advantages of Best-First Search

1. Faster than BFS and DFS (in many cases)

- It searches the **most promising path first** instead of exploring every path.
- This often reduces the search time.

Example: Google Maps chooses the road that looks closest to your destination.

2. Uses Heuristics (Smart Guess)

- It uses a heuristic function to estimate which path is better.
- This helps it make smarter decisions.

3. Can Reach the Goal Quickly

- If the heuristic is good, it can find the goal in fewer steps.

Example: Searching for a classroom by following the room numbers instead of checking every room.

4. Explores Fewer Nodes

- It does not visit every node like Breadth-First Search.
- This saves time.

5. Suitable for Large Problems

- It works well when there are many possible paths, such as route planning and game playing.

Disadvantages of Best-First Search

1. Depends on the Heuristic

- If the heuristic gives poor estimates, the algorithm may choose the wrong path.

Example: Google Maps suggests a shorter-looking road, but it is actually blocked.

2. Does Not Always Find the Shortest Path

- It may find a solution quickly, but not necessarily the best or shortest one.

3. Uses More Memory

- It stores many nodes in memory while searching.
- This can become a problem for very large search spaces.

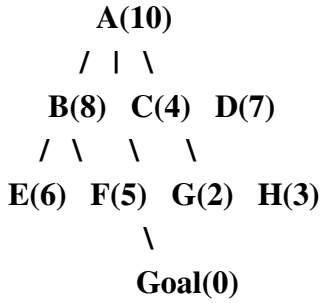
4. Can Get Trapped in a Bad Path

- It may keep following a path that looks good initially but is not the best route.

5. Performance Depends on the Problem

- If the heuristic is inaccurate, the algorithm may perform poorly.

Example



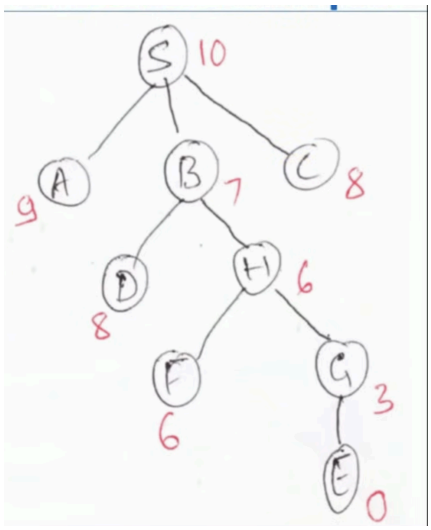
h(n) = Estimated distance (or cost) from the current node to the goal.
Smaller h(n) means the node is believed to be closer to the goal.

h(A) = 10 → Estimated distance from A to Goal = 10
h(B) = 8 → Estimated distance from B to Goal = 8
h(C) = 4 → Estimated distance from C to Goal = 4

h(G) = 2 → Estimated distance from G to Goal = 2
h(Goal) = 0 → Already at the goal

NOTE: The heuristic value is **only an estimate**. It is **not necessarily the actual shortest distance**. It is simply used to guide the search toward the goal more efficiently.

Example



Choose the node with the smallest heuristic value h(n).

Node	h(n)
S	10
A	9
B	7
C	8
D	8
H	6
F	6
G	3
E (Goal)	0

Open			Closed	
Node	H(n)		Node	Parent
S	10			

Open			Closed	
Node	H(n)		Node	Parent
A	9		S	
B	7			
C	8			

Open			Closed	
Node	H(n)		Node	Parent
B	7		S	
C	8			
A	9			

Open			Closed	
Node	H(n)		Node	Parent
C	8		S	
A	9		B	S
D	8			
H	6			

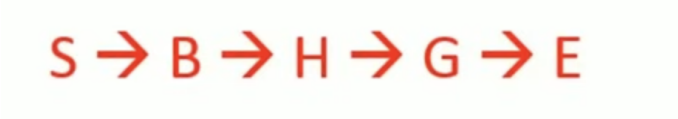
Open			Closed	
Node	H(n)		Node	Parent
H	6		S	
C	8		B	S
D	8			
A	9			

Open			Closed	
Node	H(n)		Node	Parent
C	8		S	
D	8		B	S
A	9		H	B
F	6			
G	3			

Open			Closed	
Node	H(n)		Node	Parent
G	3		S	
F	6		B	S
C	8		H	B
D	8			
A	9			

Open			Closed	
Node	H(n)		Node	Parent
E	0		S	
F	6		B	S
C	8		H	B
D	8		G	H
A	9			

Open			Closed	
Node	H(n)		Node	Parent
F	6		S	
C	8		B	S
D	8		H	B
A	9		G	H
			E	G



Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is an AI problem where we assign values to variables while following a set of rules (constraints). A solution is accepted only if all the constraints are satisfied.

Example:

```
   SEND
+ MORE
------
  MONEY
```

Variables: S, E, N, D, M, O, R, Y (letters)
Domain: Digits 0–9
Constraints: Each letter must have a unique digit, leading letters (S and M) cannot be 0, and the equation **SEND + MORE = MONEY** must be mathematically correct.

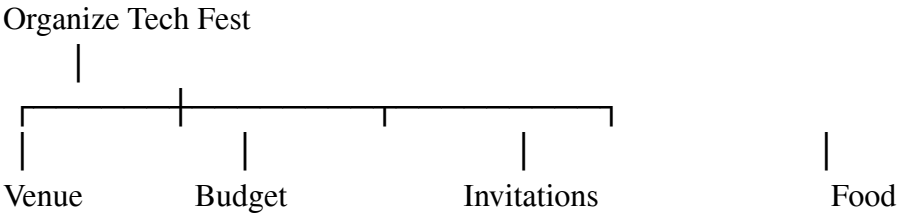
In the Cryptarithmetic Puzzle, we replace each letter with a digit while following all the rules. Since the solution is valid only when every rule (constraint) is satisfied, it is called a **Constraint Satisfaction Problem (CSP)**.

Problem Reduction

Problem Reduction is an AI technique in which a big problem is divided into smaller, easier sub-problems. After solving all the sub-problems, their solutions are combined to solve the original problem

Example:
Organize a College Tech Fest

Instead of doing everything at once, divide the work into smaller tasks.



When all four tasks are completed, the **main problem is solved**.

Note: Large Problem → Small Problems → Solve Each → Combine Solutions

Means – Ends

Means–Ends is a problem-solving technique in Artificial Intelligence where the AI compares the current state with the goal state and takes an action to reduce the difference.

Problem: AI wants to move a box from Room A to Room B.

Current State: Box is in Room A
Goal State: Box should be in Room B
Difference: Box is in the wrong room
Means/Action: Pick up the box → Move to Room B → Put down the box
End: Box reaches Room B ✅

Applications

Means–Ends Analysis is used where AI needs to **reduce the difference between the current state and the goal state**.

Application	Explanation
🤖 **Robot Planning**	Robot selects actions to reach a target or complete a task.
🗺️ **Route Planning**	Finds actions/routes that move closer to the destination.
🧩 **Puzzle Solving**	Solves problems such as the 8-puzzle by reducing differences from the goal state.
🎮 **Game Playing**	AI chooses moves that bring it closer to winning.
🩺 **Expert Systems**	Identifies differences between the current situation and desired solution.
📅**Planning & Scheduling**	Selects actions to complete tasks and achieve objectives.
🔧**Automated Problem Solving**	Breaks a large problem into smaller sub-problems.

Problem Spaces and Search: Defining the Problem as a State Space Search, Production Systems, Problem Characteristics, Production System Characteristics, and Issues in the Design of Search Programs.

Problem Space Search is the process of moving from the initial state to the goal state by trying different possible actions.

Imagine you want to go from **Home** to **College**.

- **Start (Initial State):** Home
- **Goal State:** College
- **Possible Paths:** Road 1, Road 2, Road 3

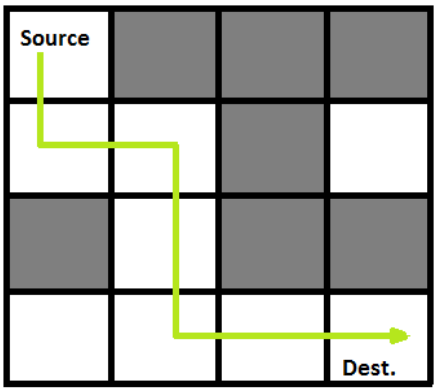
Artificial Intelligence (AI) is designed to solve problems just like humans.

Whenever AI has to solve a problem, it follows these basic steps:

1. Understand the current situation.
2. Find all possible solutions.
3. Choose the best solution.
4. Reach the goal.

This entire process is called **Problem Space Search**.

Example



Imagine you are inside a maze.

- You know where you are standing.
- You know where the exit is.

- There are many paths available.

Your task is to find the correct path. AI solves problems in exactly the same way.

What is a State? : A State is the current situation of the problem.

Example

Suppose you are in your college.
Current location = Classroom 301
This location is your state.
If you move to the library,
New location = Library
This becomes the new state.

State = Where you are right now.

What is State Space?

State space means **all possible states (positions)** that can occur while solving a problem.

Example

Suppose your college has

- Classroom
- Library
- Lab
- Canteen
- Auditorium

These are all possible locations.

Together they form the **State Space**.

In **State Space Search**, a state is represented as:

S: {S, A, Action(S), Result(S,A), COST(S,A)}

where:

- **S** is the current state,

- **A** is the selected action,
- **ACTION(S)** is the set of all possible actions from the current state,
- **RESULT(S,A)** is the new state obtained after applying the action,
- **COST(S,A)** is the cost of performing that action (such as distance, time, money, or number of moves).

Imagine you are inside a maze.

- You are standing at **(2,2)**.
- This is your **Current State (S)**.

Now you look around.

You can:

- Go Up
- Go Down
- Go Left
- Go Right

These are your **Possible Actions (ACTION(S))**.

You decide to **Go Right**.
This decision is your **Action (A)**.
After moving right, you reach **(2,3)**.
This is the **Result (RESULT(S,A))**.
You walked **one step** to reach there.
That is the **Cost (COST(S,A))**.

Symbol	Simple Meaning	Explanation	Example (Maze Problem)
S	Current State	Where you are **right now**.	You are standing at cell **(2,2)**.
A	Action	What you decide to do next.	Move **Right (East)**.
ACTION(S)	Possible Actions	All the moves you **can** make from your current position.	From (2,2), you can move **Up, Down, Left, or Right**.

RESULT(S,A)	New State	Where you reach after taking the action.	If you move **Right** from (2,2), you reach **(2,3)**.
COST(S,A)	Cost	How much it costs to perform that action. The cost can be **time, distance, number of steps, or money**.	Moving one square costs **1 step**.

Why is COST(S,A) Important?

AI often has **multiple paths** to reach the goal. The **cost** helps the AI choose the **best path**.

Example

Suppose there are two routes:

- **Path A:** 10 km
- **Path B:** 15 km

The AI compares the costs:

COST(Path A) = 10 km
COST(Path B) = 15 km

Since **10 km < 15 km**, the AI selects **Path A** because it has the **lowest cost**.

State Space Example: MAZE

State Space Search Examples: Maze

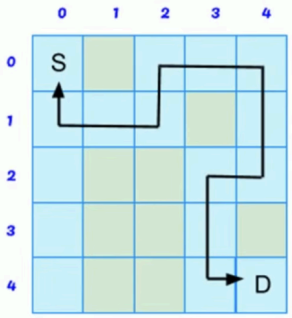
A maze problem can be represented as a state-space

A set of states

- Each state represents "where you are" that is the current position in the maze
- The start state or initial state represents your starting position
- The goal state represents the exit from the maze

Rules (for a rectangular maze) are:

- move north, move south, move east, and move west
- Each rule takes you to a new state (maze location)



State Space Search Examples: The 15 Puzzle

The 15 Puzzle problem can be represented as a state-space

A set of states

- The start is some (almost) random configuration of the tiles and
- the goal state is as shown.

Rules (for the 15 Puzzle) are:

- Move empty space up
- Move empty space down
- Move empty space right
- Move empty space left
- These rules apply if empty space is not against the edge.

Start state:

3	10	13	7
9	14	6	1
4		15	2
11	8	5	12

Goal state:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Production System in AI

A Production System is a rule-based problem-solving method used in Artificial Intelligence (AI). It solves a problem by applying a set of IF–THEN rules.

A Production System is a rule-based method in AI that solves problems by checking a condition (IF) and performing the corresponding action (THEN). It keeps applying rules until the goal is reached.

Example: **Phone Battery Charging**

Current Fact (Working Memory)

Battery = 15%

Rules (Knowledge Base)

Rule 1

IF Battery <20%
THEN Charge Phone
Rule 2

IF Battery =100%
THEN Stop Charging

Example:Classroom Attendance

Rules
IF Student is Present
THEN Mark Attendance
IF Student is Absent
THEN Mark Absent

Current Situation
Student = Present

Output : Attendance Marked

Applications of Production Systems

- Medical diagnosis systems
- Banking fraud detection
- Expert systems
- Game AI
- Route planning
- Automated planning and scheduling

Types of **Production Systems**

1. Monotonic Production System

A monotonic production system only adds new facts. It never removes or changes old facts.

Imagine you are writing notes in your notebook. Every time your teacher tells you something, you only add new notes. You never erase what you have already written. So, your notebook keeps growing. This is called a Monotonic Production System.

Example

Teacher says:

- Birds have wings. ✅ (Write it down)
- Birds can fly. ✅ (Write it down)
- Eagles are birds. ✅ (Write it down)

Nothing is removed.

The information only increases.

Example

Medical diagnosis

- Fever → Suspect infection
- Blood test positive → Bacterial infection

The system keeps adding information.

2. Non-Monotonic Production System

A non-monotonic production system can change or remove previous facts when new information is received.

Example
Now imagine Google Maps.
At 9 AM it says
Go through Road A.
After 10 minutes, there is an accident.
Google Maps now says
Don't use Road A. Use Road B instead.
It **changes** its previous decision.

This is a **Non-Monotonic Production System**.

Example

Morning

- Clouds → Predict rain

Afternoon

- Strong sunshine → Cancel rain prediction

The old decision changes because of new information.

Example

Weather forecasting

3. Partially Commutative Production System

Rules can be applied in different orders, but the final answer remains the same.

Imagine you are going from home to school.
You have two routes.
Route 1
Home → Market → School
Route 2
Home → Park → School
The roads are different.
But both reach the same school.
This is a **Partially Commutative Production System**.

Example

You want to wear shoes and a watch before leaving home.
Option 1
Wear shoes → Wear watch
Option 2
Wear watch → Wear shoes
The order changes.
Final result is the same.

Example

8-Puzzle problem
Different moves may reach the same goal.

4. Commutative Production System

The order of applying rules does not affect the process or the final result.

Concept

Imagine adding numbers.
2 + 3 + 5
You can do
(2+3)+5
or
2+(3+5)
Answer is always 10.
Everything stays the same.
This is a Commutative Production System.

Example

You are packing
Notebook
Pen
Water bottle
Whether you pack the notebook first or the bottle first,
your bag contains the same items.

Summary

Type	Idea	Real-Life Example
Monotonic	Only add information	Writing notes in a notebook
Non-Monotonic	Change previous decision	Google Maps changing the route
Partially Commutative	Different paths, same destination	Going to school by different roads
Commutative	Order doesn't matter at all	Adding numbers or packing a bag

Problem Characteristics : Describe what kind of problem we are solving.

1. Can the problem be decomposed?

Can one big problem be divided into **small independent problems**? If a problem can be divided into smaller tasks, it is called **decomposable**.

Example

College Project

Instead of doing everything together, divide it into:

- Introduction
- Literature Review
- Design
- Testing
- Conclusion

Each part can be completed separately.

2. Can solution steps be ignored or must they be remembered?

Sometimes only the **final answer** is important.
Sometimes every **step** is important.

Some problems need only the final answer, while others need the complete path.

Example 1 (Final Answer)

Calculator

2 + 5 = 7

Nobody asks how you calculated it.
Only the answer matters.

Example 2 (Steps Matter)

Google Maps

You must know

Home → Bus Stop → Highway → College

Here every step is important.

3. Is the solution reversible?

Can you **undo your action** if it is wrong?

Reversible means we can go back.
Irreversible means we cannot go back.

Example

While typing in MS Word
You type a wrong word.
Press **Undo (Ctrl + Z)**
You return to the previous state.
This is **Reversible**.
Now think of sending a WhatsApp payment.
Once the payment is completed, you cannot undo it.

This is **Irreversible**.

4. Is all information available?

Does the AI know everything before solving the problem?

Example

Chess
Both players can see every piece.
Everything is known.
Complete Information

Now Poker
You cannot see the opponent's cards.
Some information is hidden.
Incomplete Information

5. Is the solution predictable?

Will the same action always produce the same result?

Example

Traffic lights - Green always follows red on a fixed cycle.
Predictable.

Not Predictable

- **Dice roll** - Six-sided die, each face has a 1-in-6 chance, but you can't know which.
- **Lottery draw** - Numbers are random; past draws don't influence future ones.

6. Does the problem require interaction?

Does the computer solve everything alone?

Or does it need user input?

Example

Spam Filter
Works automatically.
No user interaction.

ATM Machine

Needs

- Card
- PIN
- Amount

User interaction is required.

Production System Characteristics : Describe how the AI solves the problem using rules, working memory, and a control strategy.

1. Production Rules

These are the instructions the computer follows.

They are written as

IF condition → THEN action

Example

IF Marks ≥ 40

THEN PASS
IF Marks < 40
THEN FAIL
These are production rules.

2. Working Memory

Working memory stores the current information.

Think of it as the computer's temporary notebook.

Example

Student Name : Rahul
Marks : 78
Attendance : 85%
This information is stored in working memory.
The rules use this information.

3. Control Strategy

Sometimes many rules are applicable. Who decides which rule should be applied first? That is the job of the Control Strategy.

Example

Suppose these rules exist.
Rule 1
IF fever
THEN infection
Rule 2
IF fever AND cough
THEN COVID test
The patient has
✔ Fever
✔ Cough
Both rules can be used.
The control strategy decides which rule should be execute first.

SUMMARY

Characteristic	Meaning	Example
Production Rules	IF–THEN instructions	IF Marks ≥ 40 → PASS
Working Memory	Stores current facts	Student marks
Control Strategy	Chooses which rule to apply	Teacher deciding PASS/FAIL

Issues in the Design of Search Programs

Issues in the Design of Search Programs are the important points to consider while designing an AI search program. These issues help the AI system find the correct solution quickly and efficiently. They include deciding how to represent the problem, how to search for the solution, which path to choose, and how to save time and memory.

1. Representation of the Problem

Before solving a problem, the computer must understand **what the problem is** and **what information is given**.

Example:
Imagine your teacher asks you to find **Roll No. 25** in the attendance register.

Before searching, you need:

- The attendance register
- Roll numbers
- Student names

This is **representing the problem**.

2. Search Strategy

Search strategy decides **how to search** for the answer.

Example:
You want to find your friend in a shopping mall.

You have two choices:

- Check every floor one by one.
- Go directly to the food court because your friend is usually there.

The method you choose is the **search strategy**.

3. Control Strategy

When there are many options, control strategy decides **which option to try first**.

Example:
You enter a library with **three doors**.
Which door will you enter first?
The decision of choosing the first door is the **control strategy**.

4. Pruning the Search Space

Ignore options that are **not useful** so you don't waste time.

Example:
Your pen is missing.
You know it **cannot be inside the refrigerator**.
So you don't search there.
You search only on your study table and in your bag.
This is **pruning**.

5. Heuristic Information

Use a **hint or experience** to find the answer faster.

Example:
You cannot find your TV remote.
Your mother says,
"Check under the sofa."
Instead of searching the whole house, you look under the sofa first.
This hint is called a **heuristic**.

6. Goal Test

Check whether you have found the required answer.

Example:
You are looking for your water bottle.
When you find it on the table, you stop searching.
This checking is called the **goal test**.

7. Efficiency

Find the correct answer using **less time and less effort**.

Example:
Two students solve the same math problem.

- Student A takes **20 minutes**.
- Student B uses a shortcut and finishes in **5 minutes**.

Student B's method is **more efficient**.

SUMMARY

Issue	Meaning	Real-Life Example
Representation	Understand the problem first.	Finding a student using the attendance register.
Search Strategy	Decide how to search.	Looking for a friend in a mall.
Control Strategy	Choose which option to try first.	Choosing one of three library doors.
Pruning	Ignore unnecessary options.	Not searching for your pen inside the refrigerator.
Heuristic	Use a hint or experience.	Looking under the sofa for the TV remote because someone suggested it.
Goal Test	Check if you found the answer.	Stop searching after finding your water bottle.

Efficiency	Solve the problem quickly with less effort.	Using a shortcut to solve a math problem faster.